
Module 08: Deployment

Ship your SvelteKit application to the world with optimized builds, SEO, and cloud hosting.

Revolution Trading Pros - Web Dev Course

Overview

You have built an amazing application - now it is time to share it with the world. In this final module, you will learn how to build your application for production, configure adapters for different hosting platforms, manage environment variables securely, optimize for SEO, and deploy to Vercel and Netlify.

Building for Production

The build command compiles your Svelte components, optimizes your code, and produces a production-ready output.

Terminal

```
# Build the application
pnpm build

# Preview the production build locally
pnpm preview
```

The build process performs several optimizations:

- Tree-shaking: Removes unused code from your bundle.
- Minification: Compresses JavaScript and CSS for smaller file sizes.
- Code splitting: Creates separate chunks for different routes, loaded on demand.
- Asset hashing: Adds content hashes to filenames for optimal caching.
- SSR: Pre-renders pages on the server for faster initial loads.

Analyzing Your Build

Terminal

```
# Install the bundle analyzer
pnpm add -D rollup-plugin-visualizer

# Build with analysis
pnpm build

# The analyzer generates an HTML report you can open in your browser
```

Keep an eye on your bundle sizes. A good target is under 100KB of JavaScript for the initial page load. Use dynamic imports for heavy libraries that are not needed immediately.

SvelteKit Adapters

Adapters transform your SvelteKit application for different deployment targets. The adapter determines how your app is served - as a Node.js server, serverless functions, static files, or edge functions.

adapter-auto (Default)

The auto adapter detects your deployment platform and selects the appropriate adapter automatically. It works with Vercel, Netlify, and Cloudflare Pages out of the box.

JavaScript

```
// svelte.config.js
import adapter from "@sveltejs/adapter-auto";

export default {
  kit: {
    adapter: adapter()
  }
};
```

adapter-node

For traditional Node.js servers (VPS, Docker, Railway, Fly.io):

Terminal



```
pnpm add -D @sveltejs/adapter-node
```

JavaScript

```
// svelte.config.js
import adapter from "@sveltejs/adapter-node";

export default {
  kit: {
    adapter: adapter({
      out: "build",
      precompress: true // Generate .gz and .br files
    })
  }
};
```

Terminal

```
● ● ●  
# Run the production server  
node build/index.js  
  
# Or with environment variables  
PORT=3000 HOST=0.0.0.0 node build/index.js
```

adapter-static

For fully static sites (GitHub Pages, S3, any static host):

Terminal

```
● ● ●  
pnpm add -D @sveltejs/adapter-static
```

JavaScript

```
// svelte.config.js  
import adapter from "@sveltejs/adapter-static";  
  
export default {  
  kit: {  
    adapter: adapter({  
      pages: "build",  
      assets: "build",  
      fallback: "404.html" // SPA fallback  
    })  
  }  
};
```

Note: Static sites cannot use server-side features like `+page.server.ts`, form actions, or hooks. All data must be loaded at build time or client-side.

adapter-vercel

For Vercel-specific features like edge functions and ISR:

Terminal

```
● ● ●  
pnpm add -D @sveltejs/adapter-vercel
```

```
// svelte.config.js
import adapter from "@sveltejs/adapter-vercel";

export default {
  kit: {
    adapter: adapter({
      runtime: "nodejs20.x", // or "edge"
      regions: ["iad1"],      // Deploy to specific regions
      split: true             // Split into separate functions
    })
  }
};
```

Environment Variables

SvelteKit provides a secure system for managing environment variables. There are two categories: public (exposed to the browser) and private (server-only).

Defining Variables

Shell

```
# .env file (never commit this!)

# Private - only accessible on the server
DATABASE_URL=postgresql://user:pass@localhost:5432/mydb
STRIPE_SECRET_KEY=sk_live_abc123
JWT_SECRET=super-secret-key-here

# Public - accessible in the browser (prefixed with PUBLIC_)
PUBLIC_API_URL=https://api.example.com
PUBLIC_SITE_NAME=My Awesome App
PUBLIC_GA_ID=G-XXXXXXXXXX
```

Using Variables in Code

TypeScript

```
// Server-only (in +page.server.ts, +server.ts, hooks)
import { DATABASE_URL, STRIPE_SECRET_KEY } from "$env/static/private";
import { env } from "$env/dynamic/private";

// These ONLY work in server-side files
const db = connectToDatabase(DATABASE_URL);
const stripe = new Stripe(env.STRIPE_SECRET_KEY);

// Public (anywhere, including browser)
import { PUBLIC_API_URL, PUBLIC_SITE_NAME } from "$env/static/public";
import { env } from "$env/dynamic/public";

const apiUrl = PUBLIC_API_URL;
const siteName = env.PUBLIC_SITE_NAME;
```

Important Security Rules

❗ NEVER put secrets in PUBLIC_ variables - they are sent to the browser.

- %I Add .env to your .gitignore file. Create a .env.example with placeholder values.
 - %I Use \$env/static/ when possible for better tree-shaking and build-time validation.
 - %I Use \$env/dynamic/ when variables might change at runtime (e.g., Docker containers).
 - %I Set environment variables in your hosting platform's dashboard, not in committed files.
-

SEO Optimization

Search Engine Optimization ensures your application is discoverable by search engines. SvelteKit's server-side rendering gives you a huge advantage for SEO out of the box.

svelte:head for Meta Tags

Svelte

```
<!-- src/routes/blog/[slug]/+page.svelte -->
<script lang="ts">
  let { data } = $props();
</script>

<svelte:head>
  <title>{data.post.title} | My Blog</title>
  <meta name="description" content={data.post.excerpt} />

  <!-- Open Graph (Facebook, LinkedIn) -->
  <meta property="og:title" content={data.post.title} />
  <meta property="og:description" content={data.post.excerpt} />
  <meta property="og:image" content={data.post.image} />
  <meta property="og:type" content="article" />

  <!-- Twitter Card -->
  <meta name="twitter:card" content="summary_large_image" />
  <meta name="twitter:title" content={data.post.title} />
  <meta name="twitter:description" content={data.post.excerpt} />
  <meta name="twitter:image" content={data.post.image} />

  <!-- Canonical URL -->
  <link rel="canonical" href="https://mysite.com/blog/{data.post.slug}" />
</svelte:head>

<article>
  <h1>{data.post.title}</h1>
  <!-- ... -->
</article>
```

Structured Data (JSON-LD)


```
<svelte:head>
  {@html `<script type="application/ld+json">
    {
      "@context": "https://schema.org",
      "@type": "Article",
      "headline": "${data.post.title}",
      "author": {
        "@type": "Person",
        "name": "${data.post.author}"
      },
      "datePublished": "${data.post.date}",
      "image": "${data.post.image}"
    }
  `}
</svelte:head>
```

SEO Checklist

- %ï Every page has a unique <title> and <meta name="description">.
 - %ï Open Graph and Twitter Card meta tags for social sharing.
 - %ï Canonical URLs to prevent duplicate content issues.
 - %ï Proper heading hierarchy (one h1 per page, followed by h2, h3, etc.).
 - %ï Semantic HTML throughout (header, main, article, section, footer).
 - %ï Image alt attributes for accessibility and image search.
 - %ï Fast page loads (under 3 seconds) - SvelteKit helps with this.
 - %ï Mobile-friendly responsive design.
 - %ï sitemap.xml and robots.txt in the static/ directory.
-

Deploying to Vercel

Vercel is the easiest way to deploy SvelteKit applications. It provides automatic SSL, global CDN, serverless functions, and preview deployments for every pull request.

Step 1: Push to GitHub

Terminal

```
# Initialize git (if not already done)
git init
git add .
git commit -m "Initial commit"

# Create a GitHub repo and push
gh repo create my-app --public --source=. --push

# Or add an existing remote
git remote add origin https://github.com/username/my-app.git
git push -u origin main
```

Step 2: Connect to Vercel

- %i Go to vercel.com and sign in with your GitHub account.
- %i Click 'Add New Project' and import your GitHub repository.
- %i Vercel auto-detects SvelteKit and configures the build settings.
- %i Add environment variables in the Vercel dashboard under Settings > Environment Variables.
- %i Click 'Deploy' and wait for the build to complete.

Step 3: Configure Custom Domain (Optional)

- %i Go to your project settings in Vercel.
- %i Navigate to Domains and add your custom domain.
- %i Update your DNS records as instructed by Vercel.
- %i SSL certificates are provisioned automatically.

Terminal



```
# Install Vercel CLI for advanced workflows
pnpm add -g vercel

# Deploy from command line
vercel

# Deploy to production
vercel --prod
```

Deploying to Netlify

Netlify is another excellent choice for hosting SvelteKit applications, with great support for serverless functions, form handling, and identity management.

Step 1: Install the Netlify Adapter

Terminal

```
pnpm add -D @sveltejs/adapter-netlify
```

JavaScript

```
// svelte.config.js
import adapter from "@sveltejs/adapter-netlify";

export default {
  kit: {
    adapter: adapter({
      edge: false,          // Use serverless functions (not edge)
      split: false          // Bundle all routes into one function
    })
  }
};
```

Step 2: Add netlify.toml

TOML

```
# netlify.toml
[build]
  command = "pnpm build"
  publish = "build"

[build.environment]
  NODE_VERSION = "20"
```

Step 3: Deploy

% Push your code to GitHub.

% Log in to app.netlify.com and click 'Add new site' > 'Import an existing project'.

% Select your GitHub repository.

%I Netlify detects the netlify.toml and configures the build.

%I Add environment variables under Site settings > Environment variables.

%I Click Deploy and wait for the build to complete.

Terminal



```
# Install Netlify CLI
pnpm add -g netlify-cli

# Deploy from command line
ntl deploy

# Deploy to production
ntl deploy --prod
```

Module 8 Exercise

Exercise: Deploy Your Application

Put what you have learned into practice with this hands-on exercise.

- %I Step 1: Ensure your application builds without errors by running `pnpm build`.
- %I Step 2: Preview the production build locally with `pnpm preview` and test all pages.
- %I Step 3: Create a `.env.example` file documenting all required environment variables.
- %I Step 4: Add comprehensive SEO meta tags to your home page and at least one other page.
- %I Step 5: Create a `sitemap.xml` in the `static/` directory listing all your routes.
- %I Step 6: Create a `robots.txt` in `static/` that allows all crawlers.
- %I Step 7: Push your code to a GitHub repository.
- %I Step 8: Deploy to Vercel or Netlify following the steps above.
- %I Step 9: Test the live deployment: check all routes, forms, and data loading.
- %I Step 10: Share your deployed URL with the community and celebrate!

Summary

- %I `pnpm build` creates an optimized production build with tree-shaking and code splitting.
- %I Adapters configure your app for different hosting platforms (Node.js, static, Vercel, Netlify).
- %I Environment variables are split into private (server-only) and public (browser-safe) categories.
- %I SEO requires proper meta tags, Open Graph data, structured data, and semantic HTML.
- %I Vercel provides zero-config deployment for SvelteKit with automatic adapter detection.
- %I Netlify requires the `adapter-netlify` package and a `netlify.toml` configuration file.

Congratulations!

You have completed all 8 modules of the Revolution Trading Pros Web Development Course. You now have the knowledge and skills to build, style, and deploy modern web applications with SvelteKit 5.

Here is a summary of what you have learned:

%İ Module 1: Development environment setup with Node.js, pnpm, VS Code, and Git.

%İ Module 2: HTML5 semantics, CSS Flexbox/Grid, scoped styles, and responsive design.

%İ Module 3: Modern JavaScript including arrow functions, array methods, async/await, and modules.

%İ Module 4: Svelte 5 runes (\$state, \$derived, \$effect, \$props), snippets, and control flow.

%İ Module 5: SvelteKit routing, layouts, server-side data loading, and dynamic routes.

%İ Module 6: Form actions, server-side validation, progressive enhancement.

%İ Module 7: Advanced styling, Svelte transitions, GSAP animations, and ScrollTrigger.

%İ Module 8: Production builds, adapters, environment variables, SEO, and deployment.

Keep building, keep learning, and share what you create. The web development community is welcoming and always eager to see new projects. Good luck on your journey!